Visualization of Vector Fields Using Seed LIC and Volume Rendering

Anders Helgeland, *Member*, *IEEE*, and Oyvind Andreassen

Abstract—Line Integral Convolution (LIC) is a powerful texture-based technique for visualizing vector fields. Due to the high computational expense of generating the 3D textures and the difficulties of effectively displaying the result, LIC has most commonly been used to depict vector fields in 2D or over a surface in 3D. Here, we propose new methods for more effective volume visualization of three-dimensional vector fields using LIC: 1) We present a fast method for computing volume LIC textures that exploits the sparsity of the input texture. 2) We propose the use of a shading technique, called limb darkening, to reveal the depth relations among the field lines. The shading effect is obtained simply by using appropriate transfer functions and, therefore, avoids using expensive shading techniques. 3) We demonstrate how two-field visualization techniques can be used to enhance the visual information describing a vector field. The volume LIC textures are rendered using texture-based rendering techniques, which allows interactive exploration of a vector field.

Index Terms—Vector field visualization, line integral convolution, direct volume rendering, multifield visualization.

1 Introduction

Vector fields play an important role in science and engineering. They allow us to describe a wide variety of phenomena like fluid flows and electromagnetic fields. Large vector fields often exhibit complex structures, which can be difficult to reveal using standard visualization techniques. Visualization of vector fields is one of the current challenges in scientific visualization and has been an active field of research for some years. A number of sophisticated methods have been proposed to tackle this problem.

Traditionally, vector data has been represented by arrows and other geometric primitives indicating vector magnitude and direction. More advanced methods include the display of field lines, stream surfaces [1], and flow volumes [2]. However, large three-dimensional vector fields can be difficult to visualize effectively using these techniques. Both arrows and field lines can produce cluttered and confusing images if placed densely in space. Furthermore, the limited number of field lines that can be displayed without cluttering the image makes the visualization strongly dependent on the choice of "seed points," which are the start positions of the integrated lines. It is not obvious how to distribute the field lines in space without missing important details of the field. By using texture-based techniques [3], [4], [5], some of these problems can be avoided.

A powerful texture-based visualization method is Line Integral Convolution (LIC), proposed by Cabral and Leedom [3]. Based on blurring of an input texture locally along the field lines of a given vector field, the method generates images that convey the directional information of the field. Since being introduced in 1993, it has become a popular technique and has been used to visualize both experimental data [6] and data generated from numerical simulations. A number of extensions have been proposed to increase the usefulness of the original LIC method, including improvements in speed [7], [8], [9], [10], [11], animation of steady and unsteady flow fields [12], [13], [14], [15], use of color to highlight flow features [16], extension to curvilinear surfaces [17], and extension to 3D [18], [19], [20]. Due to the high computational expense of generating the 3D textures and the difficulties of effectively displaying the result, LIC has most commonly been used to depict vector fields in 2D or over a surface in 3D. In this paper, we propose and study new methods and strategies for more effective volume visualization of three-dimensional vector fields using LIC.

First, we present a new method for computing 3D or volume LIC textures called *Seed LIC*. This method exploits the sparsity of an input texture by calculating field lines and computing the convolution starting from a set of distributed points (seed points) only. The seed points can be chosen utilizing certain properties of the field to be visualized, for example, the vector magnitude or magnitude of the curl. Sparse input textures have been used together with LIC to reveal the interior structures of a vector field [18].

Next, we propose the use of a shading technique called *limb darkening* to reveal the 3D shape and depth relations among the field lines traced by Seed LIC. The halo effect created by limb darkening is obtained simply by using appropriate transfer functions (TFs) and, therefore, avoids using expensive shading techniques.

Then, we demonstrate how texture-based direct volume rendering together with volume LIC can be used to display additional information about related scalar quantities when visualizing vector fields. One outcome of this work is a two-field visualization technique to depict the directional information of a vector field inside a Region Of Interest (ROI)

- *A. Helgeland is with the University Graduate Center, Kjeller, Norway. E-mail: ahe@ffi.no, andershe@unik.no.*
- *O. Andreassen is with the Norwegian Defence Research Establishment, Kjeller, Norway. E-mail: oya@ffi.no.*

Manuscript received 26 July 2003; revised 10 Feb. 2004; accepted 10 Feb. 2004.
For information on obtaining reprints of this article, please send e-mail to: tvcg@computer.org, and reference IEEECS Log Number TVCG-0063-0703.

1077-2626/04/$20.00 © 2004 IEEE Published by the IEEE Computer Society

defined by a scalar field. We use this technique to depict the directional structures of the vorticity field within vortices obtained from a turbulent wind-shear simulation [21], [22].

Due to rapid improvements in graphics hardware technology, a number of hardware-accelerated methods for flow visualization have been proposed recently that take advantage of the latest hardware features. Since some of these techniques are able to generate visualizations of flow fields at interactive rates, we end this paper with a comparison between Seed LIC and hardware-based methods.

2 The Seed LIC Method

In this section, we describe a new method for computing volume LIC textures called Seed LIC. The computation of the Seed LIC texture can be divided into two steps:

1. Generation of an appropriate sparse input texture masked by a Region Of Interest (ROI) function.
2. Applying Seed LIC to the input texture.

2.1 Generation of an Appropriate Input Texture

When LIC is applied to a solid 3D input texture, the output is a solid 3D LIC texture that is blurred along the direction of the vector field. Such a solid texture can be challenging to visualize clearly due to the perceptual difficulties encountered with dense 3D displays. Interrante and Grosch [18] presented several strategies for effectively illustrating 3D flow with volume LIC. This includes strategies to define an appropriate input texture by using ROI functions to highlight important features of a vector field and sparse input textures [23] to render a collection of densely placed field lines instead of a solid object. Since texture-based techniques allow the display of a large number of lines simultaneously in an image, the positioning of individual field lines becomes less important. This allows for the use of statistical methods for distributing the seed points. We adopt these strategies when generating the input texture.

We have tried different approaches for distributing the seed points in the 3D texture. In the first approach, the idea was to make a texture where the points were distributed after the scalar value that was used in making the ROI mask. Hence, we get output images where regions with high scalar values are more emphasized than other regions with lower values. As a consequence, the regions with the highest scalar values become more cluttered than the regions with lower scalar values. Another option is a more random approach. This method leads to an LIC texture where the field lines are more evenly distributed. Fig. 1 shows some examples of LIC applied to input textures generated with different distribution functions using the same region of interest. In Fig. 1a, the points in the input texture were distributed according to the vorticity magnitude, while, in Fig. 1b and Fig. 1c, a random approach was used. The visualized vector field shows the vorticity field obtained from a simulation of shock waves [24]. The input textures $(594 \times 394 \times 194)$ were defined to be twice as large as the vector field so that the details could be more easily seen.

The algorithm we have used for computing a random input texture masked by a region of interest can be described by the pseudocode in Fig. 2. The region of interest is specified by a scalar quantity, such as vorticity magnitude or temperature, while the density of the seed points is determined by the *density factor*. The final set of points chosen defines the input texture and is set to 255. The rest of the voxels are set to zero. To differentiate the strokes in the output texture, the use of white noise data has been common when applying LIC to a dense input texture. When applying LIC to a sparse input texture though, the use of various levels of gray are not necessary [23]. Instead, we differentiate the individual field lines by employing a shading technique called limb darkening. This will be discussed in Section 5.

Consistent with [18], we obtained the best results when restricting the distributed points in the input texture from

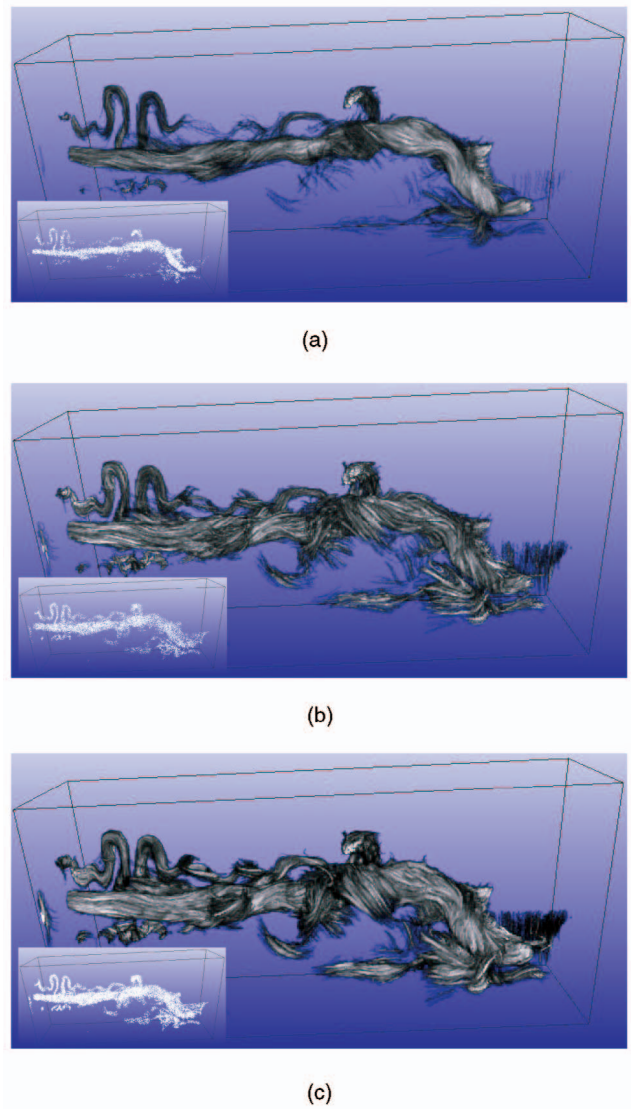

Fig. 1. LIC applied to input textures generated with different distribution functions. The small images in the left corners show the input textures. The vorticity magnitude was used to specify the region of interest. (a) The input texture where 9,514 seed points are distributed after the vorticity magnitude and the resulting LIC texture. (b) The input texture where 9,528 seed points are distributed randomly (using the value 0.065 as the density factor) and the resulting LIC texture. (c) The input texture where 20,724 seed points are distributed randomly (using the value 0.14 as the density factor) and the resulting LIC texture.

```
for each voxel v
    set input texture value to zero
for each voxel v
    if (scalar value(v) ∈ ROI) then
        compute random number ([0,1])
        if (random number > density factor ([0,1]))
            input texture value(v) = 255
            (throw away point if its to close to another seed point)
```

Fig. 2. Pseudocode for a random input texture masked by an ROI.

residing too close to each other. This is done by throwing away any point that falls within a specified minimum distance of a previously inserted point. Fig. 4 shows the result after applying LIC to a sparse input texture with a minimum distance between the seed points. To prevent the lines from getting too close, ideally, the distribution of the field lines itself should be controlled, rather than the distribution of points [4]. However, by limiting the total length of the field line, reasonable results are obtained by just controlling the position of the seed points.

Finally, we would like to mention that the scalar values chosen to define the region of interest varies from simulation to simulation and acquire some insight in the data to be visualized.

2.2 Seed LIC

For large 3D data sets (512^3 or greater), even the Fast LIC algorithm proposed by Stalling and Hege [7] is very computationally intensive. Instead of using the Fast LIC algorithm as in [18], we propose a new method that exploits the sparsity of the input texture.

When using a sparse input texture that has been masked by a region of interest, most of the voxels in the 3D input texture are set to zero. When using the Fast LIC algorithm on a texture like this, many unnecessary computations are performed in areas that make no contribution to the final image. In Fast LIC, a sufficient number of field lines are calculated so that every voxel in the output texture is covered, correlating the voxel values along these lines. But, in cases where the field lines do not coincide with cells in the input texture that contain the seed points (i.e., those points initialized to 255), no additional information is propagated to the voxels along them. The value in those voxels will be the same in the input and output texture (which is zero) and the computations in these cells are unnecessary. By only calculating field lines from the voxels containing the seed points, the number of computationally intensive operations is reduced.

The algorithm, which we have called Seed LIC, is based on Fast LIC and can be described by the pseudocode in Fig. 3, where I is the intensity for a voxel located at $\mathbf{x}_0 = \boldsymbol{\sigma}(s_0)$ given by the convolution integral

$$I(\mathbf{x}_0) = \int_{s_o-L}^{s_0+L} k(s - s_0)\, T(\boldsymbol{\sigma}(s))\, ds.$$

In this equation, k denotes the filter kernel of length $2L$ and T denotes the input texture. The curve $\boldsymbol{\sigma}(s)$ is parameterized by the arc-length s. The main difference from Fast LIC is that, in Seed LIC, we initiate the field lines and compute the convolution starting from the seed points only. In addition, the intensity of the voxels is normalized so that the voxel values vary from 0-255, increasing the range of the data. All voxels are set to zero as initial values.

```
for each voxel v
    set voxel value to zero
for each voxel v
    if (input texture value(v) > 0) then
        initiate field line computation with x_0 = center of v
        compute convolution I(x_0)
        add result to voxel
        set m = 1
        while m < some limit M
            update convolution I(x_m) and I(x_-m)
            add result to voxels containing x_m and x_-m
            set m = m + 1
for each voxel v
    normalize intensity according to numHits(v)
for each voxel v
    normalize intensity so that highest value is 255
```

Fig. 3. Pseudocode of Seed LIC.

TABLE 1
Computational Time for Fast LIC and Seed LIC

Resolution	h_t	n	Fast LIC	Seed LIC
$64 \times 64 \times 64$	0.5	4943	58.62 s	3.65 s
$128 \times 128 \times 128$	0.25	10298	11+ m	15.38 s
$192 \times 192 \times 192$	0.167	13305	51+ m	29.76 s
$256 \times 256 \times 256$	0.125	14993	2+ h	45.75 s

The computational time using Fast LIC and Seed LIC. The value h_t is the distance between different sample points and n is the number of seed points in the input texture.

3 Results

To compare the computational time of the two algorithms, we have implemented both Fast LIC[1] and Seed LIC and applied the two techniques to a synthetic vector field of resolution $64 \times 64 \times 64$. Textures of various resolutions were generated by adjusting the size of the input texture by a factor of 1, 2, 3, and 4 times the vector field size. The LIC textures were computed using a field line length of 51 times the size of the grid cells and the length of the filter kernel or the *convolution length* was set to 21 times the grid cell size. The computations were done on an SGI ONYX 3800 using a single 500 MHz R14000 processor. Table 1 shows that Seed LIC reduces the computational cost for sparse input textures immensely.

Fig. 4 shows the results after applying the two algorithms on a sparse input texture of resolution $398 \times 298 \times 798$. While the Fast LIC texture took about 25 hours to compute, the

1. Both Fast LIC and Seed LIC have been implemented using a fifth-order adaptive Runge-Kutta method with a fixed sample interval.

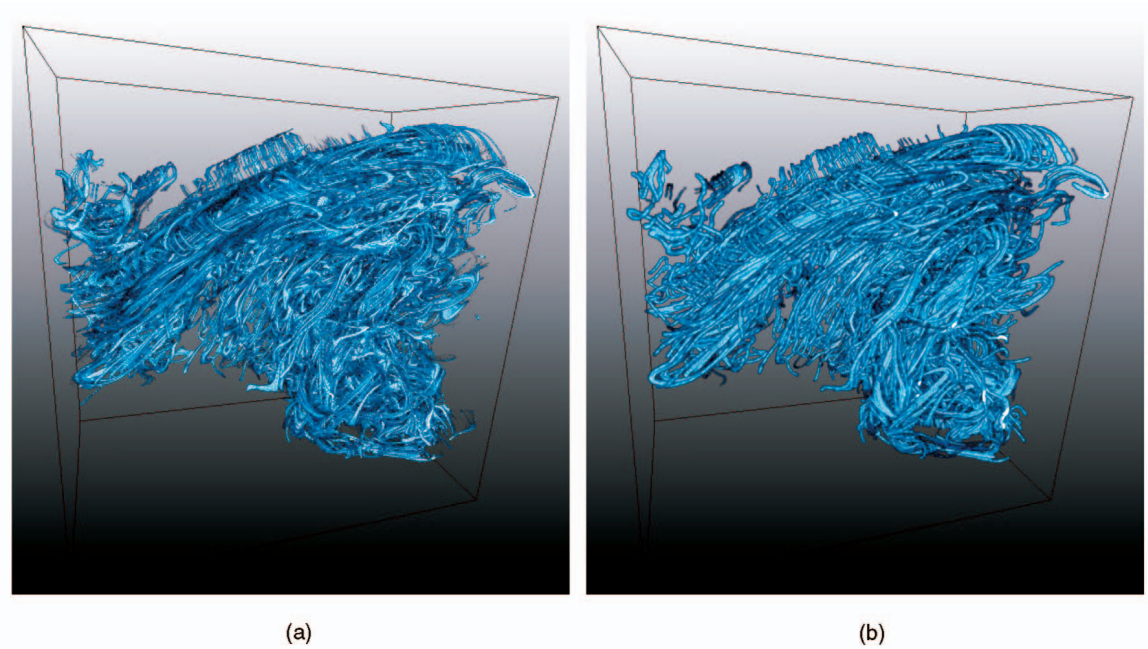

Fig. 4. (a) Visualization of a vorticity field ($\omega = \nabla \times \mathbf{u}$) using Fast LIC (computational time: 25 hours). (b) Visualization of a vorticity field using Seed LIC and convolution (computational time: 233.05 seconds). The input texture used by both Fast LIC and Seed LIC was generated using the algorithm described in Fig. 2, using vorticity magnitude to specify the ROI. To prevent the field lines from residing too close to each other, we used a minimum distance between the initial seed points. This was done by throwing away all the points that falls within three voxels of a previously inserted seed point. The input texture was defined to be the same size as the input vector field. Both LIC textures were rendered using VoluViz [25]. The transfer function used to render the Seed LIC texture is shown in Fig. 8a.

Seed LIC only took 26.86 seconds. To improve the 3D perception of the field lines and to reduce aliasing present in the Seed LIC texture, the texture has been convolved with an isotropic $3 \times 3 \times 3$ filter. The additional convolution took 206.19 seconds to compute. The convolution operation is very easy to parallelize and, for practical purposes, we have an OpenMP version which strongly reduces the computational time. Three-dimensional convolutions can also be accelerated using modern graphics hardware [26], making it possible to perform this postprocess step, even for large data sets, at nearly interactive rates. The convolution filter will be further discussed in Sections 4 and 5. The visualized vector field depicts half of the domain of the vorticity field obtained from a simulation of turbulent wind-shear [21], [22]. The simulation was conducted with $1,200 \times 400 \times 2,400$ spectral modes and reveals the breakdown of a single Kelvin-Helmholtz (KH) billow. Due to the huge amount of data derived from direct numerical simulations, it is a challenge to reveal complex structures when visualizing such flows. In some applications, it can be sufficient to study, for example, the velocity field, but, when it comes to turbulent flows, the instantaneous velocity field tends to be very complex and difficult to study. The vorticity field depicts the structure in a turbulent flow better than the velocity field due to the fact that vortices occur as elongated and coherent structures [27]. This behavior of vorticity in turbulent flows reveals the intermittent nature of turbulence. Because of this, vorticity magnitude is often used to specify a region of interest to highlight important features of the flow. The Seed LIC method proposed in this paper exploits the sparsity of the input texture. Such a technique is especially useful when used in conjunction with a ROI, as in the example above.

All of the images in this paper were created with Viz [28] and VoluViz [25]; both 3D volume-rendering tools were developed at NDRE (Norwegian Defence Research Establishment).

4 Aliasing

Aliasing can be a problem when using voxel graphics. Representing a field line with voxels, as in volume LIC, results typically in a "stair-stepped" appearance. Some of the aliasing present in the output texture is removed by trilinear interpolation during rendering.[2] This is since LIC applied to a sparse input texture generates a dense collection of field lines that are mostly surrounded by voxels containing zero values and interpolations between these values and the core values result in smoother field lines. Fig. 5a demonstrates the effect of the interpolation. The figure shows that trilinear interpolation not only reduces aliasing, but also, with an appropriate transfer function, creates a halo effect which makes it easier to separate the individual lines from one another. The halo effect will be further discussed in Section 5.

Seed LIC is more affected by aliasing than Fast LIC and therefore does not give quite as smooth a result as Fast LIC. This is due to the fewer number of voxels calculated in the Seed LIC algorithm. While Seed LIC only assigns values to the voxels along the field lines from the seed points, Fast LIC also assigns values to neighboring voxels, which leads to a

2. Both Viz and VoluViz support trilinear interpolation.

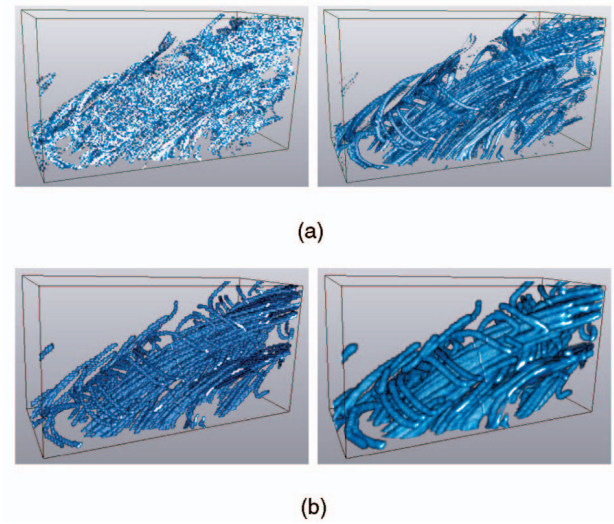

Fig. 5. The effect of trilinear interpolation and convolution filter. (a) The effect of trilinear interpolation. Left: A rendered LIC texture without the use of trilinear interpolation. Right: A rendered LIC texture with trilinear interpolation. Both the LIC textures are computed with Fast LIC. Trilinear interpolation leads to smoother field lines. The displayed texture is a small subset of the LIC texture depicted in Fig. 4a. (b) Left: The result after applying Seed LIC only. Right: The result after convolving the output texture obtained from Seed LIC. Convolving the LIC texture leads to smoother field lines. The displayed texture is a small subset of the LIC texture depicted in Fig. 4b.

smearing of the field lines. To reduce aliasing present in the Seed LIC textures, we propose a method which involves convolving the LIC texture by an isotropic $3 \times 3 \times 3$ filter. This leads to a smearing of the field lines, resulting in a smoother representation. Fig. 5b shows both the results obtained from Seed LIC and the combined Seed LIC and convolution technique. Convolving the output texture leads to a thicker field line representation. To avoid having the field lines "merge" into bundles, the initial seed points have to be restricted from residing too close to each other. If preserving the thinness of the individual field line is desired, this can be achieved by increasing the resolution of the input texture by a factor of three prior to the LIC computation.

5 Shading

Shading is important for the spatial interpretation of scientific data. Different methods have been used for the rendering of volume LIC textures and for revealing 3D flow structures. Interrante and Grosch [18] proposed a shading method which involves the use of gaps to highlight the depth discontinuities. In this method, two LIC textures are computed: one for the field lines and one for an enclosing set of halos. The final image is rendered during ray casting volume rendering, taking both textures as inputs. Another approach involves the use of the illumination model for field lines [4] and was proposed in [20]. In this paper, we propose the use of a shading technique called *limb darkening* to reveal the 3D shape and depth relations among the field lines traced by Seed LIC.

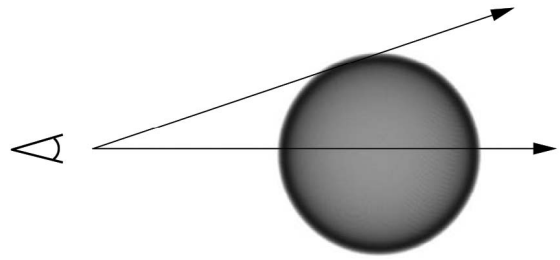

Fig. 6. When viewing the center of the sun, we see deeper into its atmosphere where the temperature is higher than in the layers we see closer to the edge. Since higher temperatures are visible as brighter, we observe a relative darkening effect at the limb.

5.1 Limb Darkening

While geometric objects have reflective properties so that the use of light sources emphasizes the 3D shape, a volume could be thought of as emitting light, where the emitted light expresses the data value of a particular voxel. Without the use of light sources, one could then anticipate that volume rendering would result in "flat" images with little information about the depth relations in the volume. By using a shading or "halo" technique called *limb darkening* [24], we can generate images with a three-dimensional appearance. The technique has its name because the effect obtained is similar to what is well-known in astrophysics. Looking at the sun with a small telescope, it is evident that the center is brighter than the limb. This is due to the fact that, when viewing the center of the sun, we see deeper into its atmosphere where the temperature is higher than in the layers we see closer to the edge. Since higher temperatures are visible as brighter, we observe a relative darkening effect at the limb (see Fig. 6).

Limb darkening can be utilized in volume visualization. By assigning darker values and decreasing the opacity near the edges of an "object" we obtain a deep three-dimensional rendering. This is illustrated in Fig. 7a. The halo effect created by limb darkening is obtained just by using appropriate TFs. If we use the HSVA color model, we can control the "temperature" or the *intensity* of the light

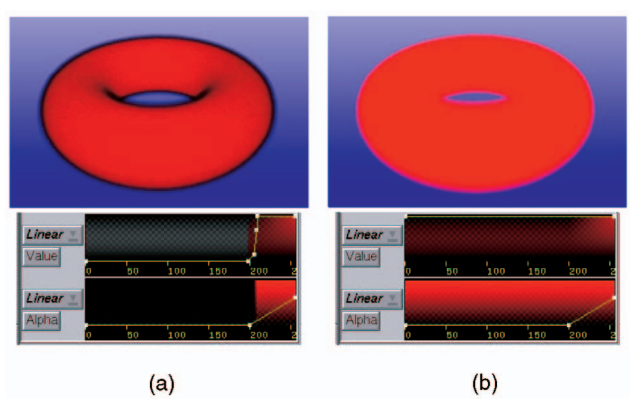

Fig. 7. Limb darkening used to visualize a torus. (a) By letting *alpha* change from zero to higher opacity values while the *value* V ranges from dark to bright, we obtain a deep three-dimensional rendering. (b) In contrast, if a constant *value* ($V = 1$) is applied to the data, a flatter appearance results.

emitted by *value* V in the HSVA color table and *opacity* through the *alpha* table. Fig. 7b shows the result after applying a constant *value* to the data. This gives a flat rendering. An effect like limb darkening is harder to control using RGBA. It should be mentioned that limb darkening is not a halo technique in the traditional sense. While a halo is a dark opaque region near the silhouettes, limb darkening creates a dark semitransparent layer all around the object. As a consequence, the nonsilhouette regions of the objects are rendered "darker" than they would be using a traditional halo technique.

Fig. 8 demonstrates the effect of applying the limb darkening technique to a volume LIC texture. To emphasize the halo effect, the textures are convolved with an isotropic $3 \times 3 \times 3$ filter. This approach smears the field lines outward, making the lines in the output texture thicker and smoother, improving the 3D perception of the LIC texture.

6 LIC and Volume Visualization

Interactivity is important for understanding complex 3D vector field structures. In order to properly study a three-dimensional vector field, the user must be able to rotate the volume, zoom in and out, and pick subsets at an interactive rate. Texture-based direct rendering [29], [30] allows interactive visualization and manipulation of 3D data.

In order to ensure comprehensive and convenient exploration of vector data, Rezk-Salama et al. [19] suggest interactive functionalities such as transfer functions and clip planes. Interactive manipulation of transfer functions simplifies the process of finding appropriate color and opacity values and ensures the quick production of a meaningful visualization of a volume LIC texture, including limb darkening. Since the halo effect created by limb darkening is obtained simply by manipulating TFs, more expensive shading techniques are avoided that could interfere with the interactivity. The use of clip planes allows the exploration of the interior structures of a vector field. By interactively moving a clip plane inside a volume, the user is able to follow the field lines in a volume LIC texture more clearly. This significantly improves the spatial understanding of the vector field. The best results are achieved when using clip planes in conjunction with LIC applied to sparse input textures.

7 Two-Field Visualization

Texture-based direct volume rendering is very useful when studying multiple fields at the same time and there exist several methods for doing this. The rendering of two fields can, for example, be achieved by using two independent sets of color and opacity tables. Merging the two data sets is done by applying a double set of sampling planes, one for each field. This is useful when comparing two fields. Another approach is to let one of the fields define the structure or the "body" through opacity and the other field to define the color. This method is useful for conveying information about related scalar quantities obtained from a simulation.

Fig. 9 shows an example of the first approach by visualizing two fields simultaneously: *vorticity* ($\omega = \nabla \times \mathbf{u}$), represented by the LIC texture, and *enstrophy* ($\eta = \|\omega\|^2$). The visualized data are obtained from [21], [22] and show a small subset of the solution revealing some vortex structures. To study details in the subset, the input texture was oversampled by a factor of 3 in each dimension. This leads to textures with higher resolution, enabling the rendering of finer and more accurate field lines. The enstrophy field of the whole KH billow at an early instant can be seen in Fig. 11. Fig. 10 shows the second approach to depict the directional structures of the vorticity field within the vortices. In this method, which we

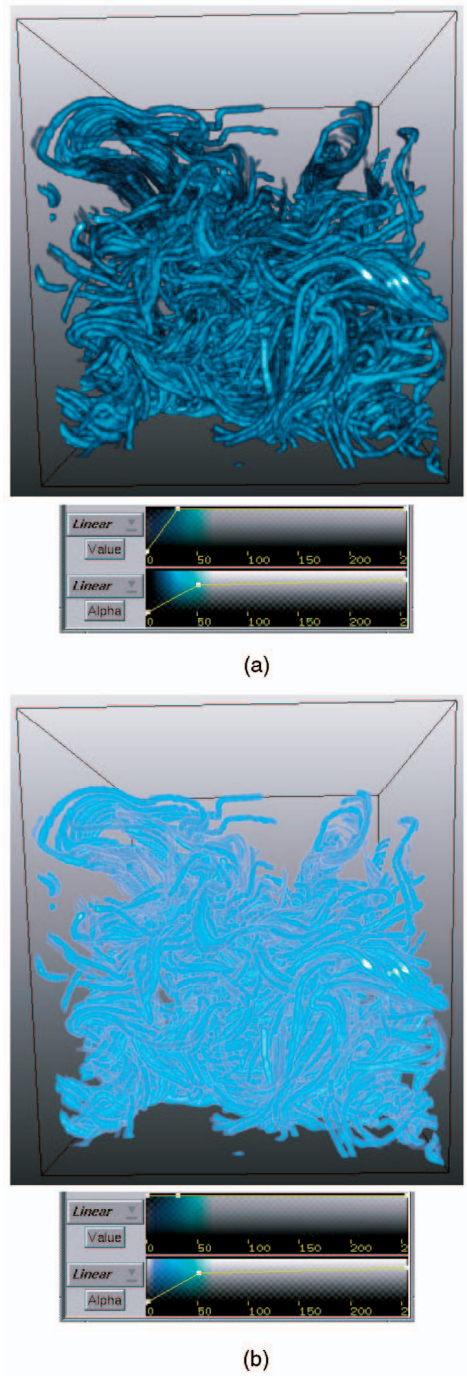

Fig. 8. Limb darkening used to visualize field lines in a Seed LIC texture. (a) By letting *alpha* change from zero to higher opacity values while the *value* V ranges from dark to bright, we obtain a deep three-dimensional rendering of the field lines. (b) In contrast, if a constant *value* ($V = 1$) is applied to all the voxels in the LIC volume, a flatter appearance results.

Fig. 9. Top: Visualization of the enstrophy field. The color varies from yellow to red with increasing enstrophy value. Middle: Visualization of the vorticity field using Seed LIC. Bottom: Two-field visualization, using two independent sets of color and opacity tables. Both fields are displayed simultaneously.

have called "candy cane" visualization (or polkagris visualization—in Norwegian), the enstrophy is used to define the opacity while the LIC texture displaying the vorticity field is used to define the color. Since enstrophy expresses the vortical structures of the flow, we get images of vortex tubes

Fig. 10. Two-field visualization using enstrophy to define the opacity and the vorticity field (represented by the Seed LIC texture) to define the color. The field lines of the vorticity field twist around the vortices, resulting in features reminiscent of "candy canes."

colored by the LIC texture conveying the directional structure of the vorticity field. The field lines of the vorticity field twist around the vortices, resulting in "objects" similar to "candy canes" (see Figs. 10 and 12). The TFs which leads to the visualizations in Figs. 10 and 12 are shown in Figs. 13 and 14.

8 Seed LIC versus Hardware-Accelerated Methods

As the focus in graphics hardware technology is shifting from higher performance implementation of the traditional graphics pipeline toward additional functionality, a number of hardware-accelerated methods for flow visualization have been proposed that take advantage of the added hardware features. Even though most of these techniques are restricted to 2D flows [10], [31], [32], [33], [34], some work has been done to also incorporate 3D flows [35], [36], [37]. As most of these techniques are able to generate real-time visualizations of steady and unsteady vector fields, it is of interest to compare these methods against our proposed Seed LIC method. Since Seed LIC is an instantaneous method, the discussion is limited to visualization of field lines in a three-dimensional vector field and includes

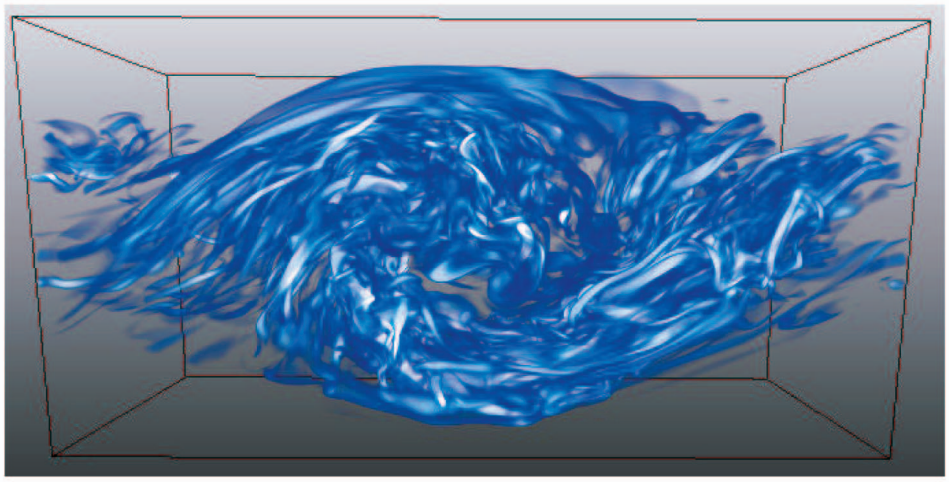

Fig. 11. Visualization of the enstrophy field of a Kelvin-Helmholtz billow at an early instant [21], [22].

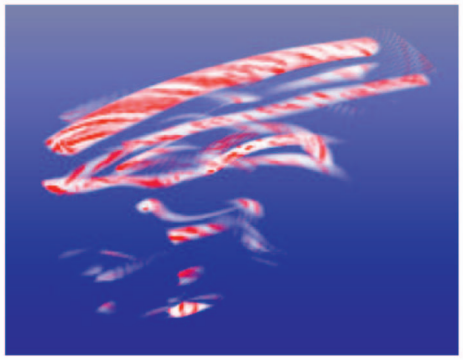

Fig. 12. Candy cane visualization.

methods by Weiskopf et al. [35], Telea and van Wijk [36], and Li et al. [37].

The first two methods are similar hardware-accelerated texture advection techniques to visualize the motion of particles in 3D flows. Even though the purpose of these techniques is to generate interactive flow animations, both can be used to generate field lines in a steady vector field by the injection of dye. Conversely to our work, which uses a fifth-order Runge-Kutta method, the algorithms proposed by Weiskopf et al. and Telea and van Wijk only use Euler integration to generate the field lines. Fig. 15 shows the result after using the two integration methods to generate LIC images displaying field lines in a synthetic vector field of resolution $50 \times 50 \times 30$. While the Euler method seems to work well for the outer helix, we see that the inner helix differs from the curve generated by the Runge-Kutta method when using the same step size. As a result, areas with high curvature may be visualized incorrectly using the methods by [35] and [36]. In order to generate field lines similar to those depicted in Fig. 15a using the Euler method, we had to reduce the step size by a factor of 20 (see Fig. 15c). In addition to the loss of accuracy by using a first-order integration method, both algorithms are affected by the limited accuracy on the GPU, restricting the resolution of the vector field to the resolution of the color channels. Another issue concerning the work by [35] and [36] is the lack of shading, which we believe to be important for the spatial understanding of the vector field. Even though the method proposed by Telea and van Wijk addresses the issue of depth occlusion by using "opacity noise" combined with alpha testing, little is done to differentiate the

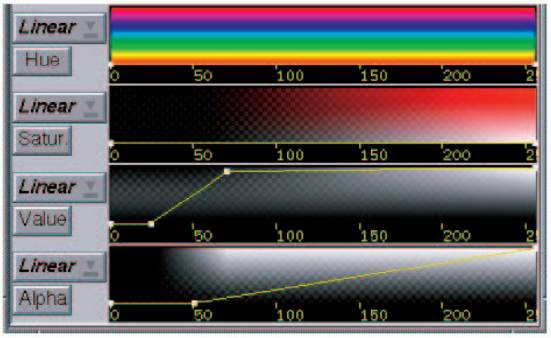

Fig. 13. The transfer function which leads to the visualization in Fig. 10.

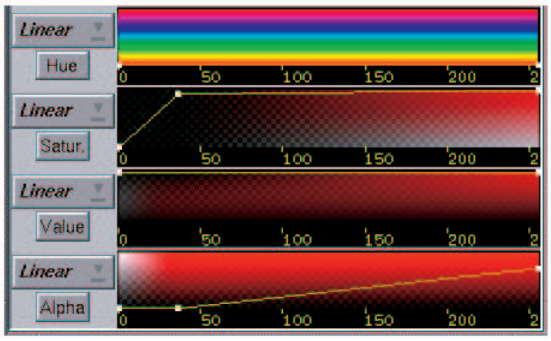

Fig. 14. The transfer function which leads to the visualization in Fig. 12.

individual field lines. In light of this, we believe that Seed LIC is more suited for visualizing field lines in large three-dimensional vector fields which exhibit complex structures, like the wind-shear simulation [21], [22] used in this paper.

The third technique, by Li et al. [37], is a technique that has much more in common with the Seed LIC method than the mentioned texture advection techniques, as both are instantaneous methods for visualizing three-dimensional vector fields. One important aspect, by which the *Chameleon* technique by Li et al. differs from our work, is how the output texture to be rendered is generated. While the field lines and voxel intensities in the output texture are calculated simultaneously by Seed LIC, the *Chameleon* technique splits this process into two stages. First, field lines are generated from the 3D vector field. Various geometric properties of the field lines are then extracted and converted into a volumetric form through a voxelization process to generate a *trace volume*. In the second phase, the trace volume is combined with a desired *appearance texture* to create various visual representations of the vector field. As the field line generation is the only step in this algorithm without any hardware-assistance, this ensures real-time rendering when modifying the appearance texture.

One important difference between the two algorithms is in the distribution of the seed points. While Li et al. use the method by [38] to select seed points randomly over the entire domain or a user-specified ROI geometry, the input texture in the Seed LIC method is often masked by an ROI function (see Fig. 1). This strongly reduces the number of field lines that have to be calculated compared to the Chameleon method. If the seed points, in Seed LIC, are distributed randomly over the whole domain, as was the case in the computations in Table 1, both methods should spend about the same time on field lines calculations assuming that the same integration method and parameter settings are used.[3] However, when using a region of interest function to highlight important features in large data sets, the Seed LIC can be much faster compared to the method by Li et al. One of the strong aspects of the Chameleon method is that, once the trace volume is created, the user can manipulate various parameters to create desired visualizations in real-time, including visual effects like shading,

3. The field line calculations in Table 1 took, on average, about 90 percent of the total computational time of the Seed LIC algorithm. Knowing this makes it possible to compare the timings of the entire LIC texture and the trace volume generation.

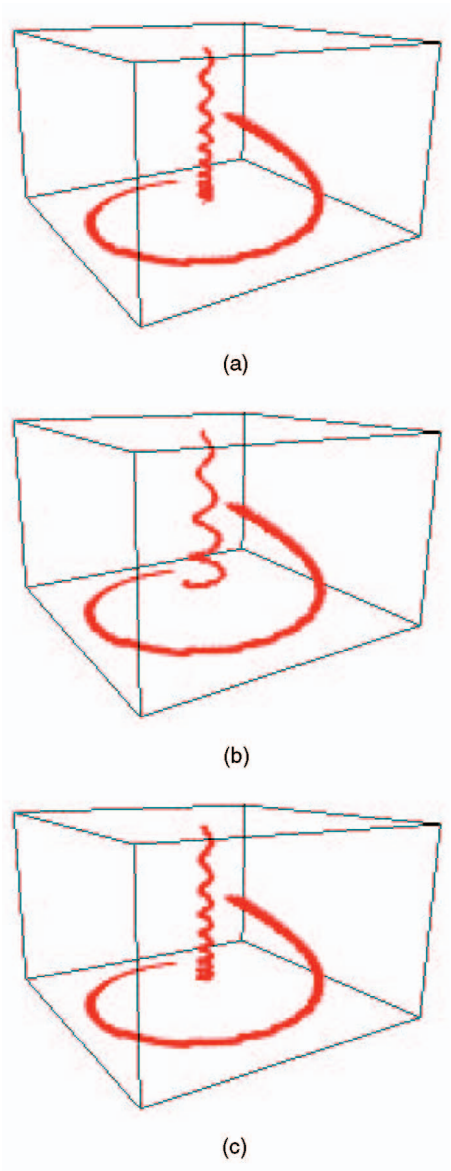

Fig. 15. Visualization of LIC textures to show the results of using two different integration methods. The LIC textures ($100 \times 100 \times 60$) were defined to be twice as large as the vector field. (a) Field lines generated by a fifth-order Runge-Kutta method using a step size of 0.25 times the grid cell size. (b) Field lines generated by a Euler method using a step size of 0.25 times the grid cell size. (c) Field lines generated by a Euler method using a step size of 0.0125 times the grid cell size.

lighting, silhouettes, etc. Unfortunately, the computation of just one trace volume is rarely enough. When scientists use volume renderer capabilities to investigate 3D flows, they often wish to study small structures, like the vortices in Fig. 9, in more detail. This requires a recomputation of the trace volume.

As in the LIC method, the Chameleon framework uses texture-based direct rendering to visualize the volumetric data sets at an interactive rate. As mentioned above, the decoupling of the visualization pipeline into two disjoint stages allows the user to experiment with different appearance textures and various parameters to create a wide range of effects at run time. The downside is the increasing demands in the graphics hardware. While our method only needs to store a single 3D luminance texture (or two luminance textures in the case of two-field visualization) in addition to the color table in texture memory, the Chameleon technique needs to store up to eight times more if both the normal vectors and the velocity magnitude are to be used. If placing the entire trace volume and normal volume in texture memory is crucial to achieving interactive rates, the Chameleon method only allows interactive rendering of volumes of size 256^3 when using graphics cards equipped with 128 Megabyte of texture memory. This means that some of the visualizations presented in this paper may be difficult to achieve using the Chameleon framework. Still, we believe that the Chameleon method is well-suited for visualizing three-dimensional vector fields since it generates, like the Seed LIC method, quality visualizations of high accuracy.

9 Conclusion

In this paper, we have proposed new methods for more effective volume visualization of three-dimensional vector fields using LIC. First, we have presented a fast method for computing volume LIC textures that can be used in an interactive setting. With an appropriate input texture, this method generates images that clearly and effectively reveal the directional structures of a vector field. To reveal the depth relation among the field lines, we have proposed the use of a shading or halo technique called *limb darkening*. To emphasize the halo effect and to reduce aliasing effects due to the use of voxels to represent the field lines, the textures are convolved with an isotropic $3 \times 3 \times 3$ filter. This approach smears the field lines outward, making the lines in the output texture thicker and smoother, improving the 3D perception of the LIC texture. To obtain a finer and more accurate representation of the field lines, the input texture can be oversampled by a factor of 2 to 4 in each dimension prior to the LIC computation. Finally, we have demonstrated how texture-based direct volume rendering, together with volume LIC, can be used to display additional information about related scalar quantities when visualizing vector fields. This is used to enhance the visual information describing a vector field. We are continuing to investigate methods for effective volume visualization of three-dimensional time-dependent flows using Seed LIC.

Acknowledgments

The authors would like to thank Mr. Trond Gaarder, Dr. Atle Ommundsen, Dr. Carl Erik Wasberg, and Dr. Bjorn Anders Reif at NDRE (Norwegian Defence Research Establishment) for their valuable contributions and feedback. They would also like to thank Dr. Joe Werne and Dr. Dave Fritts (Colorado Research Associates) and Jan Olav Langseth (NDRE) for providing the data sets and for valuable discussions.

References

[1] J.P.M. Hultquist, "Interactive Numerical Flow Visualization Using Stream Surfaces," *Computing Systems in Eng.*, vol. 1, nos. 2-4, pp. 349-353, 1990.

[2] N. Max, B. Becker, and R. Crawfis, "Flow Volumes for Interactive Vector Field Visualization," *Proc. IEEE Visualization '93,* pp. 19-24, Oct. 1993.

[3] B. Cabral and C. Leedom, "Imaging Vector Fields Using Line Integral Convolution," *Computer Graphics Proc.,* Ann. Conf. Series, vol. 27, pp. 263-270, July 1993.

[4] D. Stalling, M. Zöckler, and H.-C. Hege, "Fast Display of Illuminated Field Lines," *IEEE Trans. Visualization and Computer Graphics,* vol. 3, no. 2, pp. 118-128, Apr.-June 1997.

[5] U. Diewald, T. Preusser, and M. Rumpf, "Anisotropic Diffusion in Vector Field Visualization on Euclidean Domains and Surfaces," *IEEE Trans. Visualization and Computer Graphics,* vol. 6, no. 2, pp. 139-149, Apr.-June 2000.

[6] E.K. Longmire, B. Ganapathisubramani, I. Marusic, T. Urness, and V. Interrante, "Effective Visualization of Stereo Particle Image Velocimetry Vector Fields of a Turbulent Boundary Layer," *J. Turbulence,* vol. 4, no. 023, 2003.

[7] D. Stalling and H.-C. Hege, "Fast and Resolution Independent Line Integral Convolution," *Computer Graphics Proc.,* Ann. Conf. Series, pp. 249-256, Aug. 1995.

[8] M. Zöcker, D. Stalling, and H.-C. Hege, "Parallel Line Integral Convolution," *Proc. First Eurographics Workshop Parallel Graphics and Visualization,* pp. 249-256, Sept. 1996.

[9] V. Verma, D. Kao, and A. Pang, "PLIC: Bridging the Gap between Streamlines and LIC," *Proc. IEEE Visualization '99,* pp. 341-348, 1999.

[10] W. Heidrich, R. Westermann, H.-P. Seidel, and T. Ertl, "Applications of Pixel Textures in Visualization and Realistic Image Synthesis," *Proc. Symp. Interactive 3D Graphics,* pp. 127-134, 1999.

[11] U. Bordoloi and H.-W. Shen, "Hierarchical LIC for Vector Field Visualization," 2000, citeseer.nj.nec.com/556987.html.

[12] L. Forssell and S. Cohen, "Using Line Integral Convolution for Flow Visualization: Curvilinear Grids, Variable-Speed Animation, and Unsteady Flows," *IEEE Trans. Visualization and Computer Graphics,* vol. 1, no. 1, pp. 133-141, Mar. 1995.

[13] R. Wegenkittl, E. Gröller, and W. Purgathofer, "Animating Flow Fields: Rendering of Oriented Line Integral Convolution," *Proc. Computer Animation '97,* pp. 15-21, June 1997.

[14] H.-W. Shen and D. Kao, "UFLIC: A Line Integral Convolution Algorithm for Visualizing Unsteady Flows," *Proc. IEEE Visualization '97,* pp. 317-323, 1997.

[15] H.-W. Shen and D.L. Kao, "A New Line Integral Convolution Algorithm for Visualizing Time-Varying Flow Fields," *IEEE Trans. Visualization and Computer Graphics,* vol. 4, no. 2, pp. 98-108, Apr.-June 1998.

[16] H.-W. Shen, C.R. Johnson, and K.-L. Ma, "Visualizing Vector Fields Using Line Integral Convolution and Dye Advection," *Proc. 1996 Symp. Volume Visualization,* pp. 63-70, Oct. 1996.

[17] L. Forssell, "Visualizing Flow over Curvelinear Grid Surfaces Using Line Integral Convolution," *Proc. IEEE Visualization '94,* pp. 240-247, Oct. 1994.

[18] V. Interrante and C. Grosch, "Visualizing 3D Flow," *IEEE Computer Graphics and Applications,* vol. 18, no. 4, pp. 49-53, July-Aug. 1998.

[19] C. Rezk-Salama, P. Hastreiter, C. Teitzel, and T. Ertl, "Interactive Exploration of Volume Line Integral Convolution Based on 3D-Texture Mapping," *Proc. IEEE Visualization '99,* pp. 233-240, 1999.

[20] Y. Suzuki, I. Fujishior, L. Chen, and H. Nakamura, "Case Study: Hardware-Accelerated Selective LIC Volume Rendering," *Proc. IEEE Visualization '02,* pp. 485-488, 2002.

[21] J. Werne and D.C. Fritts, "Stratified Shear Turbulence: Evolution and Statistics," *Geophysical Research Letters,* vol. 26, no. 4, pp. 439-442 Feb. 1999.

[22] B.A.P. Reif, J. Werne, O. Andreassen, C. Meyer, and M. Davis-Mansour, "Entrainment-Zone Restratification and Flow Structures in Stratified Shear Turbulence," *Proc. 2002 Summer Program: Studying Turbulence Using Numerical Simulation Databases-IX,* pp. 245-256, Dec. 2002.

[23] V. Interrante, "Illustrating Surface Shape in Volume Data via Principal Direction-Driven 3D Line Integral Convolution," *Computer Graphics Proc.,* Ann. Conf. Series, pp. 109-116, Aug. 1997.

[24] J.O. Langseth, "3D Visualization of Shock Waves Using Volume Rendering," *Proc. Int'l Conf. Godunov Methods,* pp. 549-556, 1999.

[25] T. Gaarder and A. Helgeland, "VoluViz 1.0 report," Technical Report FFI/RAPPORT-2002/03449, FFI (Norwegian Defence Research Establishment), Sept. 2002.

[26] M. Hopf and T. Ertl, "Accelerating 3D Convolution Using Graphics Hardware Case Study," *Proc. IEEE Visualization '99,* pp. 471-474, 1999.

[27] A. Vincent and M. Meneguzzi, "The Spatial Structure and Statistical Properties of Homogeneous Turbulence," *J. Fluid Mechanics,* vol. 225, pp. 1-20, 1991.

[28] "Viz," ftp://ftp.ffi.no/spub/stsk/viz/features.html, 1998.

[29] B. Cabral, N. Cam, and J. Foran, "Accelerated Volume Rendering and Tomographic Reconstruction Using Texture Mapping Hardware," *Proc. 1994 Symp. Volume Visualization,* pp. 91-98, Oct. 1994.

[30] T.J. Cullip and U. Neumann, "Accelerated Volume Reconstruction with 3D Texture Mapping Hardware," Technical Report TR93-027, Dept. of Computer Science, Univ. of North Carolina, May 1994.

[31] B. Jobard, G. Erlebacher, and M.Y. Hussaini, "Hardware-Accelerated Texture Advection for Unsteady Flow Visualization," *Proc. IEEE Visualization '00,* pp. 155-162, Oct. 2000.

[32] D. Weiskopf, G. Erlebacher, M. Hopf, and T. Ertl, "Hardware-Accelerated Lagrangian-Eulerian Texture Advection for 2D Flow Visualization," *Proc. Vision, Modeling, and Visualization 2002,* Nov. 2002.

[33] U. Bordoloi and H.-W. Shen, "Hardware Accelerated Interactive Vector Field Visualization: A Level of Detail Approach," *Proc. Eurographics 2002, Computer Graphics Forum,* vol. 21, no. 3, pp. 605-614, Sept. 2002.

[34] J.J. van Wijk, "Image Based Flow Visualization," *Proc. 29th Ann. Conf. Computer Graphics and Interactive Techniques,* pp. 745-754, 2002.

[35] D. Weiskopf, M. Hopf, and T. Ertl, "Hardware-Accelerated Visualization of Time-Varying 2D and 3D Vector Fields by Texture Advection via Programmable Per-Pixel Operations," *Proc. Vision, Modeling, and Visualization 2001,* Nov. 2001.

[36] A. Telea and J.J. van Wijk, "3D IBFV: Hardware-Accelerated 3D Flow Visualization," *Proc. IEEE Visualization '03,* pp. 233-240, Oct. 2003.

[37] G. Li, U. Bordoloi, and H. Shen, "Chameleon: An Interactive Texture-Based Rendering Framework for Visualizing Three-Dimensional Vector Fields," *Proc. IEEE Visualization '03,* Oct. 2003.

[38] B. Jobard and W. Lefer, "Creating Evenly-Spaced Streamlines of Arbitrary Density," *Visualization in Scientific Computing '97, Proc. Eighth Eurographics Workshop,* pp. 43-56, 1997.

Anders Helgeland received the BS and MS degrees in computer science from the University of Oslo, Norway, in 2000 and 2002. He is a PhD candidate in computer science at the University Graduate Center, Norway. His current research interests include flow visualization and volume visualization. He was a recipient of the best poster award at Visualization 2002. He is a member of the IEEE and the IEEE Computer Society.

Oyvind Andreassen studied astrophysics at the University of Oslo, where he received the MS degree in 1981 and the PhD degree in physics in 1994. He is currently chief scientist at the Norwegian Defence Research Establishment and an adjunct professor in applied mathematics at the University Graduate Center at Kjeller and University of Oslo. His research interests include scientific computing and visualization, wave physics, computational fluid dynamics, turbulence, and flow noise.

▷ **For more information on this or any computing topic, please visit our Digital Library at** www.computer.org/publications/dlib.